

IT Project Guidance -

Event Source versus State based Operational System Persistence Models

Version: 0.4

Purpose

This paper provides a structured method for evaluating whether Event Sourcing is appropriate for operational systems in government and enterprise environments. It defines how the decision should be framed, how organisational impact should be assessed, and which decision traps must be avoided.

The focus is specific: whether a system should reconstruct current state from a historical event stream, rather than maintain authoritative state directly. The guidance establishes a default position of state-first, event-informed design, with Event Sourcing requiring clear justification.

Synopsis

Event Sourcing represents a shift in how systems establish truth. Instead of storing what is true now, systems store what has happened and derive the present through interpretation of history.

This paper provides a method to assess whether that shift is appropriate in an enterprise environment. It frames the decision in terms of where operational truth is anchored, evaluates the organisational capabilities and consequences, and identifies common traps that distort judgement.

Event Sourcing does not remove complexity; it redistributes it across projections, integration, reporting, and operations, increasing reliance on interpretation and specialist capability.

For the majority of enterprise systems, a state-based core with deliberate event capture remains the more sustainable approach. Event Sourcing should be reserved for domains where continuous interpretation of history is essential to service delivery *business* outcomes¹.

¹ Not to be conflated with development and diagnostics outcomes.

Contents

Purpose	1
Synopsis	1
Contents	2
Purpose and Audience	4
Scope	4
Background	5
State Based System Design	5
Event Based System Design	5
Event Persistence versus Event Streams	6
The Potential Business Value	6
Business Value of State-Based System Designs	6
Business Value proposition of Event Sourcing System Designs	7
Decision Framing	8
Step 1: What Decision Is Actually Being Made	8
Nature of the decision:	8
Anchor of business certainty:	9
Permanency of the commitment	9
Step 2: Evaluation Dimensions – Can the Organisation Sustain the Choice	9
Workforce sustainability	9
Interpretive burden as the steady state	10
Ongoing and expanding resource cost	10
Operational load and supportability	10
Integration and enterprise alignment	10
Cost displacement across the organisation	11
Opportunity Cost	11
Step 3: Decision Traps to Actively Avoid	11
Auditability trap	11
Proportionality trap	11
Sunk cost fallacy	12
Diagnosis-as-normality trap	12
Obscured business value trap	12
Relationship to Recommendation and Conclusion	12
Technical Analysis – Actual Outcomes	13
It is a permanent log, not a database	13
State is not removed, it is rebuilt elsewhere	13
Interpretation becomes mandatory, not optional	13
Replay is not safe by default	13
Capabilities already exist elsewhere in the enterprise	13
The net effect	14
Recommendation	14
Conclusion	14

Appendices	16
Appendix A - Document Information.....	16
Versions.....	16
Images.....	16
Tables.....	16
References	16
Review Distribution	16
Audience.....	17
Structure	17
Diagrams	17
Acronyms.....	17
Terms.....	17
Appendix B –	Error! Bookmark not defined.
Existing event-sourced systems	Error! Bookmark not defined.
A practical transition approach	Error! Bookmark not defined.
How to gain the benefits of events without full Event Sourcing..	Error! Bookmark not defined.
Appendix C - A technical summary of persisting events in an RDBS to achieve all of the value of Event Sourcing with less of the disruption.	Error! Bookmark not defined.

Purpose and Audience

This guidance paper provides a practical basis for evaluating Event Sourcing in government and enterprise systems. It is intended to help decision-makers, architects, delivery teams, and governance stakeholders distinguish between the genuine value of event records and the much larger architectural commitment involved in making Event Sourcing the primary persistence model for an operational service.

The purpose of this paper is not to reject events, audit trails, or event-driven integration. These techniques can be valuable. The focus is narrower and more consequential: whether an operational system should be designed so that its current state is reconstructed from an event stream, rather than held directly in a mature state-based model.

This distinction matters because Event Sourcing is often described as a modernised database or an enhanced audit mechanism. In practice, it changes the operational character of a system: how truth is established, how integrations are built, how defects are diagnosed, how change is tested, and how the system is sustained over time.

The recommended position is a state-first, event-informed approach for most government systems. Relational state remains the operational authority, while events are captured deliberately where they provide demonstrable business value.

The paper is written for a mixed audience, including architects, technical leads, developers, testers, data specialists, governance reviewers, and senior decision-makers. It assumes varied technical depth and therefore frames Event Sourcing as an organisational and full lifecycle decision, not a purely technical development pattern choice.

Scope

This paper considers the architectural decision between state-based persistence and Event Sourcing for operational government systems. It also covers how event records can be used responsibly alongside state-based systems without requiring full Event Sourcing.

The paper does not argue against event-driven architecture in general. Publishing integration events, maintaining audit logs, recording decision outcomes, and using messaging platforms can all be appropriate. The focus is on the stronger claim that an event store should be the primary source of truth, with current state derived by replay.

Background



Figure 1: Two distinct storage paradigms

The question is not whether persisting events are useful, but whether they should act as supporting records or as the authoritative persistence model.

This paper contrasts the two distinct system designs approaches.

State Based System Design

Most enterprise systems are designed around *state*. A state-based system records what is currently true within its domain: the current status of a request, the current entitlement value, the current relationship, or the current permissions held by a user. State may change over time and history of these changes may be retained, but operational behaviour centres on retrieving and updating current records.

This model is familiar because it underpins relational database systems, used by the majority² of enterprise applications. Relational platforms provide structure, constraints, transactions, relationships, migration discipline, recovery, and a large ecosystem of tooling and skills. Their strictness can slow early development, but this discipline underpins is not a liability – it is a feature. It promotes long-term stability, governance, recoverability, and supportability.

Importantly, state-based design does not imply loss of history. Well-designed systems deliberately retain audit and change records through ledgers, history tables, versioned entities, workflow transitions, and logs. Current state remains authoritative and material to business service delivery, while history supports secondary activities of assurance, diagnostics, reporting, and learning.

Event Based System Design

Event-based approaches shift attention toward recording what has happened. Event Sourcing takes this further by treating the event record itself as the source of truth. Current state is not stored directly but derived from the ordered sequence of events.

This can be valuable where the sequence of activity is itself central to the business capability. However, adopting Event Sourcing is not simply a different storage choice. It

² Relational Database Management System Market Size and Growth Report, 2033

materially changes how systems are designed, tested, integrated, diagnosed, and supported. These implications extend across the system lifecycle.

Event Persistence versus Event Streams

Within the broader context of interest in recording events, an distinction must be made between *persisting* events and adopting *sourcing* events. The concerns are related, but they represent materially different architectural aspects.

Event Persistence: a system may retain event records for audit, traceability, analytics, or integration without those events defining operational state. Likewise, systems may publish events and use asynchronous processing without being event-sourced.

Event Sourcing: is a specific architectural pattern in which the event record becomes the system of record, and current state is *derived* by replaying or projecting that history.

Event streaming may be used to support event sourcing, but streaming is an implementation detail rather than the defining characteristic.

The Potential Business Value

Business value is realised only when an architectural choice improves how the organisation operates, decides, complies, or governs. If benefits are primarily technical or diagnostic, the value is not business-facing.

The central evaluative question is:

Does this business service gain enough enduring value from event-sourced persistence to justify the long-term operational, integration, capability, and maintenance costs imposed across the organisation?

This must be assessed in contrast to what state-based systems already provide

Business Value of State-Based System Designs

State-based system design has existed since the 1950s, with relational expression and SQL formalised in the 1970s. It represents nearly three-quarters of a century of incremental evolution around atomic, consistent, isolated, and durable (ACID) transactions. This lineage is not incidental. It reflects sustained pressure from the most operationally demanding environments.

For decades, state-based systems have run airlines, airports, banks, payroll, human resources, finance, inventory, sales, and entitlement systems. Their value is not novelty, but reliability under scale, regulation, and scrutiny.

That value goes beyond data correctness. State-based systems provide **certainty of agreement about current state**. At any moment, there is a clearly authoritative answer to questions such as: What is the balance? What is the entitlement? What is the status?

What permissions apply now? This certainty is reinforced through relationships, constraints, and transactional boundaries. Changes propagate consequences immediately and visibly. Invalid states are rejected. Inconsistencies surface as errors rather than lingering ambiguities.

This shared certainty enables coordination across an organisation. Reporting systems, integration partners, financial controls, audit processes, and operational dashboards all align around the same understanding of “what is currently true”. Disruption is detectable precisely because the implications of change are structurally visible.

From a delivery perspective, the ecosystem around state-based systems is mature and replaceable. Skilled practitioners are widely available. Tooling for modelling, validation, testing, migration, performance analysis, governance, and reporting is extensive and interchangeable. Vendors can be changed within reasonable bounds. This reduces long-term organisational risk and dependency.

State-based design also does not imply loss of history. Well-designed systems retain change history deliberately, through audit tables, temporal records, ledgers, versioned entities, workflow transitions, and other structured mechanisms. In these designs, current state remains the operational authority, while history supports assurance, diagnostics, reporting, compliance, and learning. The system does not need to reconstruct what is true by replaying its entire past in order to operate.

Much of the business value of relational systems is therefore invisible precisely because it is foundational. A large, layered ecosystem of reporting, governance, assurance, and operational practice exists because stable, authoritative current state exists.

The business value of the design pattern is its predictability of being able to be achieved, cost-effectiveness.

Business Value proposition of Event Sourcing System Designs

Event Sourcing can be valuable when the sequence of events is itself a primary source of business insight or operational advantage. In some domains, understanding the path to the current state is as important as the state itself. This can be true in areas such as logistics, inventory optimisation, market activity, telemetry-heavy platforms, behavioural analytics, fraud detection, recommendation systems, and other environments where patterns over time can directly improve decisions.

In those contexts, the organisation may need to know not only that stock is low, but how it became low, how quickly demand changed, which locations were affected, which behaviours preceded the change, and what interventions improved the outcome. The value is not merely historical curiosity. The value is better decision-making.

Event Sourcing can also assist some forms of replay analysis. If the organisation wants to ask “what would have happened if the rule had been different?”, a detailed event record

can be valuable. If context changes frequently and the organisation can improve outcomes by reinterpreting past activity, then the event record can become an asset.

However, this value depends on the domain. It cannot be assumed. Recording more history does not automatically create more business value. A system can accumulate a very large and technically interesting event store without materially improving outcomes for users, administrators, policy owners, or decision-makers. Without change to business service unachievable in another way, there is no business value that can be assigned to it.

Decision Framing

This section sets out a clear and disciplined method for deciding whether Event Sourcing is an appropriate architectural choice for an operational system. It is intended to support sponsors, architects, developers, delivery teams, and governance reviewers in reaching a defensible decision. The method separates the decision into three parts: framing the decision correctly, evaluating organisational consequences, and explicitly avoiding known decision traps. This method is intended to support explicit, defensible decision making within delivery and governance processes, ensuring that architectural choices are assessed in terms of long-term organisational impact rather than short-term technical preference.

Step 1: What Decision Is Actually Being Made

Decision makers should therefore frame the question explicitly as follows:

Nature of the decision:

Choosing Event Sourcing is not a decision about tooling, databases, or modern design practice. It is a decision about where the organisation places operational truth.

In a state-based system, operational truth resides in current, governed records. The system explicitly stores what is true now: the current status, entitlement, approval, balance, or obligation. History exists to explain and justify that position. In an event-sourced system, operational truth resides in historical sequence. The system stores what has happened, and “what is true” must be reconstructed by interpreting that history.

This distinction is not stylistic. It defines how certainty is established, how agreement is reached across teams, how incidents are investigated, how integrations are built, and how the system is sustained over its lifetime.

Anchor of business certainty:

Most operational services depend on shared certainty about the present. Business users, downstream systems, auditors, and partners need a clear, inspectable answer to questions such as “what is the situation now?”. State-based systems are designed to provide this certainty directly.

Event Sourcing replaces direct certainty with interpretive certainty. Agreement depends on consistent replay logic, projection rules, event versions, and shared assumptions about meaning over time. Unless the business itself requires continuous reinterpretation of history as a normal operating capability, this trade-off reduces clarity rather than increasing value.

The framing question is therefore explicit:

Does this service require historical interpretation as an operational necessity, or does it require stable current facts with history retained for assurance and explanation?

If the latter, Event Sourcing is misaligned with the business need.

Permanency of the commitment

Once operational truth is anchored in an event stream, the organisation commits permanently to interpretation as a core capability. There is no later point at which the system becomes simple again. Every policy change, rule update, defect correction, integration, audit, and incident response must reason across historical meaning.

This is not reversible without significant re-engineering. Event Sourcing must therefore be treated as a long-term organisational commitment, not an incremental design option.

Step 2: Evaluation Dimensions – Can the Organisation Sustain the Choice

If the decision framing genuinely supports Event Sourcing, proposals must then be evaluated against concrete organisational consequences. These considerations do not determine whether Event Sourcing is conceptually valid; they determine whether it is sustainably ownable within the organisation that must build, operate, support, and evolve it over time.

Workforce sustainability

Event Sourcing requires sustained architectural judgement rather than routine application of established patterns. Skills are less common, harder to transfer, and more dependent on tacit knowledge than those required for state-based relational systems.

The organisation must assume that many future contributors will not be the original designers. If the system cannot be safely understood, operated, and evolved by future staff and vendors using broadly available skills, the architecture introduces long-term

fragility. In public sector environments, where systems persist for decades, this is a material and enduring risk.

Interpretive burden as the steady state

Event Sourcing permanently embeds interpretation as business-as-usual. Understanding current behaviour, diagnosing faults, assessing correctness, and explaining outcomes requires replaying history through evolving logic.

This burden does not reduce as the system matures. It accumulates. Complexity is not eliminated by Event Sourcing; it is normalised and distributed across development, testing, operations, and assurance.

Ongoing and expanding resource cost

Event-sourced systems grow without bound by design. Events are retained, replayed, and reinterpreted indefinitely. Over time, this increases demands on storage, compute, reprocessing, and operational tooling.

Mitigation techniques such as truncation, snapshotting, or compaction introduce additional governance and operational complexity and are frequently deferred. Cost therefore grows with system age as well as transaction volume. The correct evaluation question is not whether the system is affordable at launch, but whether it remains sustainable across its full operational lifespan.

Operational load and supportability

Event Sourcing materially changes support practices. Failures often surface indirectly, in projections, reconciliations, or downstream artefacts, rather than at the point of change. Root cause analysis is delayed and requires reasoning across time, event versions, and side effects.

There is no inherently safe replay model once external effects exist. Preventing duplication of payments, notifications, approvals, or other real-world actions depends on discipline and defensive design rather than guaranteed controls. This raises the baseline level of operational complexity and risk.

Integration and enterprise alignment

Most enterprise and government environments are state-oriented. Reporting platforms, financial systems, dashboards, identity services, and partner integrations depend on shared, stable answers to “what is true now”, and on common data skills and patterns.

Event-sourced systems must recreate these answers through projections. This does not remove the need for state; it relocates it into derived structures that must be built, governed, tested, reconciled, and maintained over time. These projections are not incidental integration artefacts. They are a core architectural cost of adopting Event Sourcing and must be treated as such.

As a result, the system risks diverging from enterprise norms and becoming a specialised enclave. Integration, reporting, and assurance functions adapt around it, rather than the system aligning with organisational practice. Substitution becomes difficult, exit paths narrow, and organisational cohesion weakens.

Cost displacement across the organisation

Even where Event Sourcing delivers local benefit, that benefit is not free. The adopting team may gain flexibility or diagnostic depth, but the cost of interpretation, projection management, reconciliation, and specialised operational handling is pushed onto the rest of the organisation.

Reporting teams, integration partners, governance functions, and operational support absorb this cost, often without receiving proportional benefit. The value is localised; the cost is socialised. Unless this redistribution is explicitly acknowledged and justified, the architecture obscures rather than reduces its true organisational cost.

Opportunity Cost

Effort invested in event mechanics, projection infrastructure, replay tooling, and specialised operational processes is effort not invested in business capability, usability, integration quality, or service improvement.

Where the business value of Event Sourcing is marginal, speculative, or indirect, this displacement becomes a material concern. Capacity is finite. Architectural effort must be judged against alternative uses that may deliver clearer and more immediate benefit.

Step 3: Decision Traps to Actively Avoid

Even with correct framing and evaluation, Event Sourcing decisions are prone to predictable distortions. These traps should be treated as governance signals.

Auditability trap

Event Sourcing is often justified on the grounds of audit. This is a misunderstanding. Strong auditability requires traceability, attribution, immutability where appropriate, and retention discipline — all of which can be achieved in state-based systems. Audit requirements alone do not justify making history the operational source of truth.

Proportionality trap

Architectures designed for highly dynamic, optimisation-driven domains are sometimes applied to stable services with predictable state transitions. This results in complexity that does not improve outcomes. Proportion matters.

Sunk cost fallacy

Past investment explains how the organisation arrived at its current architecture, but it does not justify continued investment if value is not materialising. Decisions should be based on future cost and benefit, not on the effort already expended.

Diagnosis-as-normality trap

Rich event histories can help with investigation, but a well-designed system should not depend on forensic reconstruction as part of everyday operation. If understanding system behaviour routinely requires replay and interpretation, this points to structural complexity that has been accepted rather than resilience.

Obscured business value trap

When Event Sourcing is too complex for business users to engage with meaningfully, its value ceases to be business value. At that point it functions primarily as a developer-centric diagnostic tool while imposing cost and friction on the wider organisation.

Governance erosion under “new ways of working”

While not directly related to Event Sourcing, delivery approaches framed as “new ways of working” can create conditions where new architectural guardrails are deferred or weakened under delivery pressure. This can lead to erosion in the rigour of analysing and governing key technical decisions, including those addressed in this paper.

Where oversight weakens, decisions become implicit and local practices replace shared standards. The result is fragmentation rather than agility: degraded process discipline, inconsistent quality, disruption to business outcomes, increased cost, and reduced assurance. This risk is amplified in architectures that already rely on interpretation and specialised practices.

Decision makers should treat any combination of significant architectural change as exemplified by this paper, along with reduced tested guardrails and oversight as high risk. Guardrails, constraints, and governance must be established as first steps, not after³.

Relationship to Recommendation and Conclusion

This section defines the method used by this guidance: correct framing of the decision, explicit evaluation of organisational consequences, and active avoidance of known traps. The Recommendation and Conclusion apply this method to establish a default position

³ back-filled or never

and to define clearly the narrow circumstances under which an exception may be justified.

Technical Analysis – Actual Outcomes

This section does not introduce new concepts. It describes, in practical terms, what Event Sourcing becomes when used as an operational persistence model, presenting the approach in a form aligned to familiar constructs and established enterprise practices, so that the implications of the choice can be understood without abstraction.

It is a permanent log, not a database

Event Sourcing replaces a system that stores current truth with a system that stores a permanent record of what happened. In practice, this is an application-level log — similar in form to a database write-ahead log — except it is not transient or internal, or able to be truncated. What other database systems use as transient tool to get to state, becomes the system of record and must be retained indefinitely. The system no longer answers “what is true now” directly; it reconstructs that answer from the log each time.

State is not removed, it is rebuilt elsewhere

The business still requires current state. Users, integrations, and reports all need stable answers. Event-sourced systems therefore rebuild state through projections. This does not eliminate state; it moves it out of the primary model and into derived structures that must be maintained, repaired, and kept consistent. The organisation ends up managing both the event log and the reconstructed state.

Interpretation becomes mandatory, not optional

Current truth is no longer inspectable. It depends on applying the same replay logic, event versions, and projection rules every time. Agreement about “what is true” depends on consistent sequence and interpretation, not on a shared record. This introduces a permanent interpretive burden across development, testing, operations, and assurance.

Replay is not safe by default

Replaying events is not equivalent to re-running the system. Real systems have side effects that must not be repeated. Preventing duplicate payments, notifications, or external actions depends on discipline and additional mechanisms. There is no fully safe replay model once side effects exist.

Capabilities already exist elsewhere in the enterprise

Organisations already solve “what happened over time” using data warehouses and analytical stores. These platforms reconstruct history from stable state for reporting,

audit, and analysis. Event Sourcing embeds a similar capability inside the operational system, duplicating what the enterprise already provides, but without the same maturity, tooling, or shared understanding.

The net effect

Event Sourcing does not simplify the system. It replaces direct state with permanent history, reintroduces state through projections, and makes interpretation a required part of everyday operation. The result is higher complexity and cost over time, not lower.

Recommendation

For enterprise systems, unless it is an *existential* aspect of the business service, the recommended approach is to keep state as the operational authority and record state change as history table entries.

This approach does not discard the value of events. It places them in the right proportion.

The operational service should use a structured relational model for the information it is responsible for. It should define its entities, relationships, states, transitions, constraints, and rules explicitly. It should use transactions where consistency matters. It should use schema governance and migration discipline. It should expose stable state-oriented APIs for operational use and integration.

Alongside this, the service may record events as secondary records. These events may describe important business transitions, user actions, rule outcomes, external submissions, notifications, integration exchanges, or diagnostic milestones. They can be used for audit, analytics, operational monitoring, replay analysis, warehouse enrichment, and future evaluation.

In this model, the event record is valuable, but it is not the primary operational source of truth. The service does not depend on replaying the full historical stream to know its current state. The event system supports insight; it does not dominate the architecture.

This gives the organisation a more balanced design. It preserves relational correctness and operational simplicity while still allowing event-based learning where it is justified.

Conclusion

Event Sourcing is a legitimate architectural pattern. It is also a very costly one.

Its costs are not limited to development effort and storage. They include operational complexity, specialist dependency, integration burden, replay risk, event versioning, projection management, testing complexity, workforce fragility, and long-term maintainability risk.

Enterprises and government systems should not adopt that burden unless the business value is material, enduring and clearly existentially critical, and cannot be done with stabler technology and patterns.

As per the Recommendation section, the more balanced approach is to keep relational state as the operational foundation and use events as a governed supporting capability. This allows organisations to retain history, improve diagnostics, support audit, publish integration events, and develop analytical insight without making the whole system dependent on event replay.

For most government systems, the better architecture is state-first and event-informed. That approach is more maintainable, more governable, easier to integrate, easier to support, and more aligned with long-term public sector service obligations.

Appendices

Appendix A - Document Information

Authors & Collaborators

- Sky Sigal, Solution Architect

Versions

- 0.1 Initial Draft
- 0.2 Reformatted
- 0.3 Removed unnecessary detail
- 0.4 Completed Appendices

Images

Figure 1: TODO Image **Error! Bookmark not defined.**

Tables

No table of figures entries found.

References

There are no sources in the current document.

- Martin Fowler: [Event Sourcing](#)
- [Don't Let the Internet Dupe You, Event Sourcing is Hard - Blogomatano](#)
- [Don't use Event Sourcing if you want auditability - DEV Community](#)
- [When not to use Event Sourcing? - Event-Driven.io](#)
- [Stop losing information: Event Sourcing - Novanet](#)
- <https://medium.com/@dinesharney/crud-vs-event-sourcing-comparing-two-contrasting-approaches-23e3078bd226>

Review Distribution

The document was distributed for review as below:

Identity	Notes
Amy Orr, Enterprise Data Architect	
Duncan Watson, Enterprise Architect	

Ian Bloodworth, Data Solutions Architect	
Carl Klitscher, Senior Solution Architect	
Michael Rocks, Senior Database Administrator	
Vincent Wierdsma, Lead Developer	
Michael Pocock, Business Change Advisor	
Comptess Hammond, Product Owner	

Audience

The document is technical in nature, but parts are expected to be read and/or validated by a non-technical audience.

Structure

Where possible, the document structure is guided by either ISO-* standards or best practice.

Diagrams

Diagrams are developed for a wide audience. Unless specifically for a technical audience, where the use of industry standard diagram types (ArchiMate, UML, C4), is appropriate, diagrams are developed as simple “box & line” monochrome diagrams.

Acronyms

API : Application Programming Interface.

DDD

: Domain Driven Design

GUI: Graphical User Interface. A form of UI.

ICT: acronym for Information & Communication Technology, the domain of defining Information elements and using technology to automate their communication between entities. IT is a subset of ICT.

IT : acronym for Information, using Technology to automate and facilitate its management.

UI : User Interface. Contrast with API.

Terms

Refer to the project’s Glossary.

Application Programming Interface : an Interface provided for other systems to invoke (as opposed to User Interfaces).

Capability : a capability is what an organisation or system must be able to achieve to meet its goals. Each capability belongs to a domain and is realised through one or more functions that, together, deliver the intended outcome within that area of concern.

Domain : a domain is a defined area of knowledge, responsibility, or activity within an organisation or system. It groups related capabilities, entities, and functions that collectively serve a common purpose. Each capability belongs to a domain, and each function operates within one.

Entity : an entity is a core object of interest within a domain, usually representing a person, place, thing, or event that holds information and can change over time, such as a Student, School, or Enrolment.

Function : a function is a specific task or operation performed by a system, process, or person. Functions work together to enable a capability to be carried out. Each function operates within a domain and supports the delivery of one or more capabilities.

Person : a physical person, who has one or more Personas. Not necessarily a system User.

Persona : a facet that a Person presents to a Group of some kind.

Quality : a quality is a measurable or observable attribute of a system or outcome that indicates how well it meets expectations. Examples include reliability, usability, and performance. Refer to the ISO-25000 SQuaRE series of standards.

User : a human user of a system via its UIs.

User Interface : a system interface intended for use by system users. Most computer system UIs are Graphics User Interfaces (GUI) or Text/Console User Interfaces (TUI).

Appendix B – Implementing Event Value Within a Relational System

This appendix describes a concrete approach to consider for implementing a relational design that delivers auditability, traceability, temporal analysis, and integration signalling without making events the operational source of truth.

The approach consists of four coordinated structures: authoritative state tables, state transition history, a governed business event log, and a reliable event publication mechanism.

Authoritative state model

Operational truth is held in normalised relational tables with full schema governance.

Each core entity is stored in a primary table, for example:

Request

- RequestId (PK)
- CurrentState
- ApplicantId
- CurrentAmount
- LastUpdatedAt
- LastUpdatedBy
- Version (row version / concurrency token)

Constraints must be enforced at the database level: foreign keys, check constraints for allowed states, and transactional consistency. State transitions are applied through controlled commands (stored procedures or application services), not by direct table mutation.

All operational reads, APIs, and integrations retrieve current truth from these tables.

State transition history

Every state-changing operation writes a corresponding immutable record into a transition history table within the same transaction.

RequestStateTransition

- TransitionId (PK)
- RequestId (FK)
- FromState
- ToState
- TransitionType (e.g. Submit, Approve, Adjust)
- OccurredAt (UTC timestamp)
- ActorId
- ReasonCode
- RuleVersion
- CorrelationId
- EvidenceReference (nullable)

This table is append-only. No updates or deletes are permitted.

This provides:

Clear audit trace of how the current state was reached

A queryable, structured history without replay

Direct support for audit, investigation, and assurance

Indexes should be defined on RequestId and OccurredAt to support ordered reconstruction when required.

Business event log (governed, not exhaustive)

A separate, curated event table captures only events with defined business value.

BusinessEvent

- EventId (PK)
- EventType (e.g. RequestSubmitted, FundingApproved)
- AggregateId (e.g. RequestId)
- OccurredAt
- Payload (JSON)
- SchemaVersion
- CorrelationId
- ProducedBy

Key characteristics:

Events are intentionally selected, not a full internal log

Payloads follow versioned schemas

Event types align to domain language, not technical activity

This table is also append-only and independently queryable.

Transactional event publication (outbox pattern)

To guarantee consistency between state and events, both are written within the same database transaction using an outbox table.

OutboxEvent

- OutboxId (PK)
- EventId
- EventType
- Payload
- OccurredAt
- PublishedFlag (boolean)
- PublishedAt

Process:

1. Application executes a state change
2. Within the same transaction:

- Update Request table
 - Insert into RequestStateTransition
 - Insert into BusinessEvent (if applicable)
 - Insert into OutboxEvent
3. Transaction commits
 4. A background publisher reads unprocessed OutboxEvent rows
 5. Publishes to messaging infrastructure
 6. Marks PublishedFlag = true

This ensures:

No event is published without committed state

No state change occurs without corresponding audit and event records

Analytical reconstruction (outside the operational system)

Temporal analysis is performed using downstream systems:

State transitions and events are exported (ETL or streaming) into a warehouse

Time-based analytics, trend analysis, and replay experimentation occur there

Operational system remains state-driven and performant

This avoids embedding analytical replay into the operational service.

Summary of enforcement

The discipline is enforced through:

Database constraints and append-only tables

Transactional boundaries coupling state and history

Governed schemas for events

Separation of operational and analytical concerns

This delivers most of the practical benefits attributed to Event Sourcing, without making interpretation of history the mechanism by which the system determines current truth.

Appendix C – Transition Option: Reducing Dependence on Event Sourcing

If working with an existing event sourced system design, this appendix sets out a concrete, staged approach to move from an event-sourced system to a state-first, event-informed model. It is an option to consider when an event-sourced design is creating sustainability, cost, or operational challenges. It is not a mandated approach but a structured method for reducing reliance on event-based reconstruction of state while preserving service continuity.

Step 1: Identify operational state explicitly

Extract the actual current-state concepts used by the business.

For each aggregate, define:

Current status

Current calculated values

Current decisions and outcomes

External obligations and relationships

Document these as a target relational schema. This becomes the candidate authoritative model.

Step 2: Build deterministic state projections as first-class tables

Create relational tables that materialise current state from the event stream.

RequestProjection

- RequestId (PK)
- CurrentState
- CurrentAmount
- LastEventSequence
- LastUpdatedAt

Populate using a projection processor:

for each event in EventStream ordered by sequence:

 apply transformation logic

 upsert into RequestProjection

Critical requirements:

Processing must be idempotent

Ordering must be deterministic

Projection state must be versioned

At this stage, projections are still derived, but stabilised.

Step 3: Introduce transactional ownership of state

Shift from read-only projection to authoritative state:

New operations write directly to the relational model

Event emission becomes a secondary outcome, not the source of truth

Implementation pattern:

1. Command received
2. Validate against current relational state

3. Apply state change in RDBMS within transaction
4. Record transition history
5. Emit event (via outbox)

The event stream is no longer required to determine current state.

Step 4: Freeze event stream as historical record

Stop using the event stream as a source for operational reconstruction.

Actions:

Disable replay as a regular operational process

Retain event store in read-only mode

Continue to ingest new events only as derived outputs

Event history is now evidence, not mechanism.

Step 5: Migrate integrations to state-based contracts

Existing consumers of the event stream must be transitioned.

Options:

Replace full-stream consumption with subscription to business events

Provide state-based APIs for current queries

Introduce reporting views exposing current state

This removes the requirement for external systems to interpret internal event sequences.

Step 6: Align testing and assurance to state transitions

Refactor test strategy:

Assert state before and after commands

Validate transition records

Test event emission as a side effect, not the primary outcome

This reduces reliance on sequence-based reasoning.

Step 7: Incrementally retire event-processing infrastructure

Once state ownership is stable:

Reduce projection processors to verification roles

Remove replay tooling from operational use

Simplify deployment and operational pipelines

Bounded contexts can transition independently.

Step 8: Enforce forward discipline

Prevent regression into implicit event sourcing:

All new features must write through state-first model
Event generation must follow state changes
No new dependencies on replay-based logic
Governance checkpoints should enforce this.